

Technical Documentation

May 21, 2026

Abstract

This document highlights the most critical and insightful parts of the steel defect detection implementation. Rather than a full user manual, it explains the key techniques: RLE encoding/decoding (with column-major order), the U-Net architecture with pre-trained ResNet34 backbone, the combined Dice+BCE loss function, and the training pipeline. These components represent the core of a successful segmentation solution for this competition.

1 RLE Encoding and Decoding (The Foundation)

The competition uses run-length encoding (RLE) with **column-major** ordering: pixels are numbered from top to bottom, then left to right. This is the most error-prone part of the implementation.

1.1 Decoding: RLE $\rightarrow$ Binary Mask

```
1 def rle_decode(rle_str, shape=(256, 1600)):
2     if pd.isna(rle_str) or rle_str == '1 409600':
3         return np.zeros(shape, dtype=np.uint8)
4     s = list(map(int, rle_str.split()))
5     mask = np.zeros(shape[0] * shape[1], dtype=np.uint8)
6     for start, length in zip(s[0::2], s[1::2]):
7         mask[start-1:start-1+length] = 1
8     return mask.reshape(shape, order='F') # Critical: column-major
```

Key insight: The `order='F'` parameter in `reshape` ensures Fortran (column-major) ordering, matching the competition's pixel numbering scheme.

1.2 Encoding: Binary Mask $\rightarrow$ RLE

```
1 def rle_encoder(mask):
2     pixels = mask.T.flatten() # Transpose then flatten = column-major
3     if np.sum(pixels) == 0:
4         return '1 409600'
5     pixels = np.concatenate([[0], pixels, [0]])
6     runs = np.where(pixels[1:] != pixels[:-1])[0] + 1
7     runs[1::2] -= runs[0::2]
8     return ' '.join(str(x) for x in runs)
```

Key insight: `mask.T.flatten()` transposes first (swapping rows and columns) then flattens, producing the correct column-major sequence.

1.3 Special Empty Case

The string '1 409600' represents a completely empty mask (no defects). For a $256 \times 1600 = 409,600$ pixel image, this means "from pixel 1, run 409,600 pixels" — effectively the entire image as background.

2 Dataset Classes: Bridging Images and Labels

2.1 Training Dataset

```
1 class SteelDataset(Dataset):
2     def __getitem__(self, index):
3         # Load and transform image
4         image = Image.open(img_path).convert('RGB')
5         image = self.transform(image) # -> (3, 256, 1600) tensor
6
7         # Build mask for all 4 classes
8         masks = []
9         for class_id in range(1, 5):
10            row = self.df[(self.df['ImageId']==img_id) &
11                          (self.df['ClassId']==class_id)]
12            if len(row) > 0:
13                mask = rle_decode(row['EncodedPixels'].values[0])
14            else:
15                mask = np.zeros((256, 1600), dtype=np.uint8)
16            masks.append(mask)
17        mask = np.stack(masks, axis=0) # (4, 256, 1600)
18        mask = torch.from_numpy(mask).float()
19        return image, mask, img_id
```

Key insight: Each image returns a 4-channel mask (one per defect class), even if some classes have no defects.

3 U-Net Architecture with ResNet34 Backbone

3.1 Encoder (Pre-trained ResNet34)

```
1 class UNet(nn.Module):
2     def __init__(self, n_classes=4):
3         super().__init__()
4         resnet = models.resnet34(pretrained=True)
5         self.enc1 = nn.Sequential(resnet.conv1, resnet.bn1, resnet.relu
6                                   ) # 64ch
7         self.enc2 = nn.Sequential(resnet.maxpool, resnet.layer1)
8                       # 64ch
9         self.enc3 = resnet.layer2 # 128ch
10        self.enc4 = resnet.layer3 # 256ch
11        self.enc5 = resnet.layer4 # 512ch
12        # ... decoder ...
```

Key insight: The pre-trained ResNet34 (trained on ImageNet) already understands edges, textures, and shapes — features that transfer well to defect detection.

3.2 Decoder with Skip Connections

```

1 class Up(nn.Module):
2     def __init__(self, in_ch, out_ch):
3         super().__init__()
4         self.up = nn.ConvTranspose2d(in_ch, out_ch, kernel_size=2,
5                                       stride=2)
6         self.conv = DoubleConv(out_ch * 2, out_ch) # after
7                                                     concatenation
8
9     def forward(self, x1, x2):
10        x1 = self.up(x1) # upsample
11        # Handle size mismatch
12        diffy = x2.size()[2] - x1.size()[2]
13        diffx = x2.size()[3] - x1.size()[3]
14        x1 = F.pad(x1, [diffx//2, diffx-diffx//2, diffy//2, diffy-diffy
15                       //2])
16        x = torch.cat([x2, x1], dim=1) # skip connection
17        return self.conv(x)

```

Key insight: Skip connections preserve spatial detail lost during downsampling — crucial for precise defect localization.

3.3 Final Upsampling

```

1 self.final_up = nn.ConvTranspose2d(64, 64, kernel_size=2, stride=2)
2 self.out = nn.Conv2d(64, n_classes, kernel_size=1)

```

Key insight: The encoder downsamples $32 \times (256/8 = 8?$ Wait: $256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8$). Four up-convolutions only recover $16 \times (8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128)$. An extra `final_up` restores the full $256 \times$ resolution.

4 Loss Function: Dice + Binary Cross Entropy

```

1 class Loss(nn.Module):
2     def forward(self, pred, tar):
3         bce = nn.BCELoss()(pred, tar) # Pixel-wise classification
4
5         pred_flat = pred.view(pred.size(0), -1)
6         tar_flat = tar.view(tar.size(0), -1)
7         intersection = (pred_flat * tar_flat).sum(dim=1)
8         dice = (2. * intersection + 1e-6) / (pred_flat.sum(dim=1) +
9         tar_flat.sum(dim=1) + 1e-6)
10        dice = dice.mean()
11
12        return bce + (1 - dice)

```

Key insight:

- BCE provides stable gradient for pixel-wise classification.
- Dice loss directly optimizes the evaluation metric.
- The combined loss handles severe class imbalance (defects are rare).
- The small epsilon (1×10^{-6}) prevents division by zero.

5 Training Pipeline

5.1 Data Splitting

```
1 train_ids, val_ids = train_test_split(all_img_ids, test_size=0.2,  
    random_state=7)
```

Key insight: Split by **image ID**, not by rows — ensures no data leakage between training and validation.

5.2 Training Loop

```
1 def train(model, train_loader, val_loader, epochs=10):  
2     opt = torch.optim.AdamW(model.parameters(), lr=1e-4)  
3     scheduler = ReduceLROnPlateau(opt, mode='min', patience=2)  
4  
5     for epoch in range(epochs):  
6         # Training phase  
7         model.train()  
8         for images, masks, _ in train_loader:  
9             pred = model(images)  
10            loss = Loss()(pred, masks)  
11            loss.backward()  
12            opt.step()  
13  
14        # Validation phase  
15        model.eval()  
16        with torch.no_grad():  
17            for images, masks, _ in val_loader:  
18                pred = model(images)  
19                val_loss += Loss()(pred, masks)  
20  
21        scheduler.step(val_loss) # Reduce LR on plateau  
22  
23        # Save best model  
24        if val_loss < best_val_loss:  
25            torch.save(model.state_dict(), 'best.pth')
```

Key insights:

- `model.train()/model.eval()` control BatchNorm/Dropout behavior.
- `torch.no_grad()` disables gradient computation during validation (saves memory).
- `ReduceLROnPlateau` automatically lowers learning rate when validation loss stagnates.
- Only the best model (lowest validation loss) is saved.

5.3 Inference and Submission

```
1 def predict_and_submit(model, test_loader):  
2     model.eval()  
3     results = []  
4     with torch.no_grad():  
5         for images, img_ids in test_loader:  
6             pred = model(images) # (B,4,256,1600) probabilities  
7             pred = pred.cpu().numpy()
```

```

8         for i, img_id in enumerate(img_ids):
9             for class_id in range(1, 5):
10                 mask = pred[i, class_id-1] > 0.5 # Threshold
11                 mask = mask.astype(np.uint8)
12                 rle = rle_encoder(mask)
13                 results.append({
14                     'ImageId': img_id,
15                     'EncodedPixels': rle,
16                     'ClassId': class_id
17                 })
18 pd.DataFrame(results).to_csv('submission.csv', index=False)

```

Key insight: Each test image generates **4 rows** in the submission file (one per defect class), even if no defects are detected (then RLE = '1 409600').

6 Critical Design Decisions Summarized

Table 1: Key design decisions and their rationales

Component	Rationale
Column-major RLE	Competition specification; affects all encoding/decoding
Pre-trained ResNet34	Transfer learning from ImageNet provides good feature initialization
U-Net with skip connections	Preserves spatial resolution for precise defect localization
Dice + BCE loss	Handles class imbalance + directly optimizes evaluation metric
Horizontal flip augmentation	Simple but effective; steel defects can appear mirrored
Validation split by image	Prevents data leakage between training and validation
ReduceLROnPlateau	Automatically adapts learning rate for better convergence
Best model checkpointing	Keeps the model with lowest validation loss

7 Conclusion

The code’s essence lies in correctly handling the competition-specific RLE format, building a U-Net with pre-trained encoder, using a combined Dice-BCE loss, and implementing a robust training pipeline with validation and checkpointing. These components together form a complete, production-ready solution for steel defect segmentation.